King Saud University
College of Computer & Information Science
CSC111 – Extra Lab 2 (Objects)

Lab Exercise 1

Part 1

Design a class named **Ball** class that models a moving ball. The class contains:

- Two properties **`x, y`** which maintain the position of the ball in a two-dimensional space as double values.
- Two properties **`distTraveledX`** and **`distTraveledY`** that keep the total distance traveled by current ball throughout all of its moves on both x-axis and y-axis.
- Methods **`getX`** and **`getY`** that return the current position of the ball.
- A method **`move`**, which changes **`x`** and **`y`** by the given **`xDisp`** and **`yDisp`**, respectively.
- Two methods **`getDistTraveledX`** and **`getDistTraveledY`** that return the distance traveled by the current ball throughout all of its moves on both x-axis and y-axis.

Start by drawing the UML for the class **Ball**.

```
public class Ball {
	// data members
	// define instance variables x, y, distTraveledX, distTraveledY
	/* modifier datatype variable name */
	/* modifier datatype variable name */
	/* modifier datatype variable name */
	/* modifier datatype variable name */

	/* Getter for x getX() */
	/* Getter for y getY() */

	// move x by xDisp and y by yDisp
	/* modifier */ /* return type */ move ( /* parameter xDisp */, /* yDisp */){
		// set x to xDisp and y to yDisp
		distTraveledX +=/*the difference between the old and the new position*/
		// update distTraveledY
	}
}
```

Part 2

Add the following constructor to the class:

- A default constructor that sets position to (0, 0).
- A constructor that receives two parameters newX and newY that represent the current position of the ball.

```
public class Ball {
	/* data members */

	public Ball(){
		/* set x and y to zeros */
	}

	public Ball(double newX, double newY){
		/* set x to newX */
		/* set y to newY */
	}

	/* Getter for x getX() */
	/* Getter for y getY() */

	/* move method */
}
```

Part 3

In this part, you will add static members which will help you to keep track of all the balls. Add the following static properties and methods:

- Two static properties **`totDistXAllBalls`** and **`totDistYAllBalls`** that keep the total distance traveled by all balls throughout all of the moves on both x-axis and y-axis.
- Static properties **`lastX`** and **`lastY`** which store the last position of the most recent ball that has moved.
- Two static methods **`getTotDistXAllBalls`** and **`getTotDistYAllBalls`** that return the distance traveled by all balls throughout all of the moves on both x-axis and y-axis.

```
public class Ball {
	/* data members */

	// static data members
	// define static variables totDistXAllBalls, totDistYAllBalls, lastX and lastY
	/* modifier static datatype variable name */
	/* modifier static datatype variable name */
	/* modifier static datatype variable name */
	/* modifier static datatype variable name */

	public Ball(){
		/* set x and y to zeros */
	}

	public Ball(double newX, double newY){
		/* set x to newX */
		/* set y to newY */
	}

	/* Getter for x getX() */
	/* Getter for y getY() */

	/* static getter for totDistXAllBalls getTotDistXAllBalls() */
	/* static getter for totDistYAllBalls getTotDistYAllBalls() */

	/* move method */
}
```

Part 4

In this part you will define a new method to print the information of the ball and modify the move method:

- A method **`toString`**, which returns the string **`"Ball @ (x,y)"`**.
- Modify the **`move`** method so that a ball must not move if it is going to finish its movement in a position occupied by the last moved ball.
  (Hint: use `lastX` and `lastY` to determine this. If movement is allowed then change the values of `lastX` and `lastY`).

Part 5

Write a program that does the following:

- It creates a new ball with a position (2, 2).
- Then it moves the ball by (3, -2).
- Then it moves the ball by (2, -7).
- Then it creates a new ball with a position (0,0).
- Then it moves the ball by (7, -7). Notice that the second ball must not move since it will otherwise occupy the same place occupied by the first ball.
- Then it moves the ball by (2, 4).
- Finally, it prints:
  - The last position of the two balls using the **`toString`** method.
  - The total distance traveled on both x-axis and y-axis by each ball.
  - The total distance traveled on both x-axis and y-axis by all balls.

```
public class TestBall {
   public static void main(String[] args) {
      // create a new ball (b1) with position x=2 and y=2
      // move b1 by x = 3 & y = -2
      // move b1 by x = 2 & y = -7
      // create a new ball (b2) with a position(0, 0)
      // move b2 by x = 7 & y = -7
      // move b2 by x = 2 & y = 4
      // print the two balls' info using the toString() and getDistTraveled X & Y
      // print all balls traveled distance X & Y
   }
}
```

Sample Run

```
The ball in position (0.0, 0.0) cannot move to position (7.0, -7.0)
b1:
Ball @ (7.0, -7.0)
Distance Traveled X: 5.0
Distance Traveled Y: 9.0

b2:
Ball @ (2.0, 4.0)
Distance Traveled X: 2.0
Distance Traveled Y: 4.0

Total All Balls X: 7.0
Total All Balls Y: 13.0
```

Lab Exercise 2

You have been asked by the Saudi Wildlife Authority to design a class named **Species** to manage endangered species.

Part 1

Add the following requirements to the class:

- Three data fields (properties) which are:
  - **`name`** of type **`String`** which stores name of species.
  - **`population`** of type **`int`** which stores the current population of the species and it can not be negative.
  - **`growthRate`** of type **`double`** which stores the growth rate of the current population.
- Two constructors:
  1. Default constructor with no arguments where you assign all the properties to their default values (i.e. integers to 0, doubles to 0.0 and Strings to “”).
  2. A constructor that takes new arguments to be assigned to all properties.
- A method **`readInput()`** that reads the values of data members. The method must not allow the user to enter incorrect values for the population. If this happens, it should keep asking for the correct value until the user enters it.

```java
public class Species {
	/* define the three data members name , population and growthRate */

	public Species() {
		// assign each property to its default value
	}

	public Species(/* name, population and growthRate */) {
		// assign each property to its corresponding value
	}

	public void readInput() {
		// create a new Scanner
		/* print a message prompting the user to enter
		the values: name, population and growthRate */
		name = input.nextLine();
		// read population from the user
		while(population < 0) {
			/* keep asking and reading the population from the user and tell
			him that his input for the population is invalid until he enters
			a valid input*/
		}
		// read growthRate from the user
	}
}
```

Sample Run

```
What is the species' name? Dinosaur
What is the population of the species? -200
You entered an invalid population, re-enter the population again? 200
Enter the growth rate (% increase per year): 0
```

Part 2

Now you have a class called **Species**, add the following methods to the previous class:

- A method **`writeOutput()`** that prints the data of the species.
- A method **`predictPopulation(int years)`** that returns the projected population of the species after the specified number of years (which must be a non-negative number).
- Getter methods for **`name`**, **`population`**, and **`growthRate`**.

- A method **`setSpecies(String newName, int newPopulation, double newGrowthRate)`** that sets values of the receiving object to the new data sent through parameters. The method should print an error message and exit the whole program if newPopulation is a negative number.

```
public class Species {
	/* define the three data members name , population and growthRate */

	public Species() {
		// assign each property to its default value
	}

	public Species(/* name, population and growthRate */) {
		// assign each property to its corresponding value
	}

	// readInput method

	public /* return type */ writeOutput() {
		// return a summary of the specie
	}

	public /* return type */ predictPopulation(int years) {
		// check the value to make sure that it is not a negative number
		// If not return the population after "years"
	}

	// getter methods for name, population and growthRate

	public /* return type */ setSpecies(String newName, int newPopulation, double
newGrowthRate) {
		// assign each property to its new corresponding value
		// if the newPopulation is print error and exit the program
	}
}
```

Sample Run

```
What is the species' name? Houbara Bustard
What is the population of the species? 900
Enter the growth rate (% increase per year): 0.2
After 535 years, the population of Houbara Bustard will be 2621
```

Part 3

Add the following methods:

- A method **`equals(Species otherObject)`** that compares this object to ***otherObject***. Two species objects are equal if they have the same name ignoring the letter case.
- A method **`isPopulationLargerThan(Species otherSpecies)`** that returns true if the population of this object is greater than the population of otherSpecies.
- A method **`isExtinct()`** that returns true if the population of this object is 0. Otherwise, it returns false.

Part 4

Write a test program that checks if the population of an input species chosen by the user could one day exceed that of Arabian Oryx. Given that the current population of Arabian Oryx is 1000 and its growth rate is 0.25.

- The program first reads the information of some species X.
- Then it checks if species X is extinct and if so it prints the message ”The species that you entered is extinct” and exits.
- If species X is not extinct, then it checks if species X's current population is already larger than the Arabian Oryx. If so, print the result.
- If species X has less population, then it checks if it can surpass the Arabian Oryx’s population. If it can, then it prints after how many years this will happen.
- If a user enters Arabian Oryx then you should keep asking him to enter a different species.

Name your test class **TestSpecies**. Use a new separate file for the test class.

Sample Run 1

```
What is the species' name? Dinosaur
What is the population of the species? 0
Enter growth rate (% increase per year): 0
The species that you entered is extinct.
```

Sample Run 2

```
What is the species' name? Houbara Bustard
What is the population of the species? 900
Enter growth rate (% increase per year): 0.2
Species Houbara Bustard has a slower growth rate than Arabian Oryx.
```

Sample Run 3

```
What is the species' name? African Lion
What is the population of the species? 16500
Enter growth rate (% increase per year): 0.05
The population of the African Lion is already larger than the population of Arabian Oryx.
```

Sample Run 4

```
What is the species' name? Arabian Oryx
What is the population of the species? 1000
Enter growth rate (% increase per year): 0.5
You entered the Arabian Oryx species, please enter another one:
What is the species' name? arabian ORYX
What is the population of the species? 500
Enter growth rate (% increase per year): 0.25
You entered the Arabian Oryx species, please enter another one:
What is the species' name? Blue Whale
What is the population of the species? 500
Enter growth rate (% increase per year): 0.38
After 535 years, the population of the Blue Whale will surpass that of Arabian Oryx.
```